Netaji Subhas University of Technology

New Delhi - 110031

Soft Computing

EAEPC503

Practical Record File

2026-2027

Submitted By:

Name: Yatharth Shahi

Roll No: 2024UEA6638

Branch: ECAM

Section: 2

Submitted To:

Dr. Aarti Jain

Table of Content

S.No	Topic	Date	Page No.	Sign
1	Implementation of Logic Gates using Python			
2	Fuzzy Logic based Control of Fan Speed according to Room Temperature			
3	Computation of α-Cut and Strong α-Cut of a Fuzzy Set			
4	Determination of Height of a Fuzzy Set and its Classification as Normal / Subnormal			
5	Determination of Support, Core, Boundary and Crossover Points of a Fuzzy Set			
6	Conversion of a Crisp Set into a Fuzzy Set by Assigning Membership Values			
7	Union, Intersection, Complement and Difference of Fuzzy Sets; Fuzzy Relations and Max-Min Composition			
8	Perceptron Training using the Fixed Increment Learning Algorithm			
9	Implementation of Linear Regression			
10	Implementation of Multiple Linear Regression			
11	Implementation of Hebb's Rule for the AND Gate			
12	Implementation of the Delta Rule for the OR Gate			
13	Implementation of the Travelling Salesman Problem using a Genetic Algorithm			
14	Implementation of a Single-Layer Perceptron ANN (without Backpropagation) verified using the AND Gate			

EXPERIMENT 1

Objective:

To write a Python program to implement basic and derived logic gates (AND, OR, NOT, NAND, NOR, XOR, XNOR) using logical operators.

Prerequisites:

- Set up Python interpreter (PyCharm).
- Basic knowledge of Boolean algebra and truth tables.

Theory:

Logic gates are the basic building blocks of digital circuits. Each gate performs a specific Boolean operation on one or more binary inputs and produces a single binary output. In Soft Computing, the study of classical (crisp) logic gates forms the foundation for understanding fuzzy logic systems, where truth values can lie anywhere between 0 and 1 instead of being strictly 0 or 1.

- AND: Output is 1 only when both inputs are 1.
- OR: Output is 1 when at least one input is 1.
- NOT: Output is the complement of the input.
- NAND / NOR: Complement of AND / OR respectively.
- XOR: Output is 1 when the inputs are different.
- XNOR: Complement of XOR; output is 1 when inputs are the same.

PYTHON

```
A = 1
B = 0
print("Inputs: A =", A, ", B =", B)

AND = A and B
OR = A or B
XOR = A ^ B
NOT_A = not A
NOT_B = not B
NAND = not AND
NOR = not OR
XNOR = not XOR

print("Logic Gate Outputs")
print("AND  =", int(AND))
print("OR   =", int(OR))
print("XOR  =", int(XOR))
print("NOT_A =", int(NOT_A))
print("NOT_B =", int(NOT_B))
print("NAND =", int(NAND))
print("NOR  =", int(NOR))
print("XNOR =", int(XNOR))
```

Output:

OUTPUT

```
Inputs: A = 1 , B = 0
Logic Gate Outputs
AND  = 0
OR   = 1
XOR  = 1
NOT_A = 0
```

```
NOT_B = 1
NAND = 1
NOR  = 0
XNOR = 0
```

Observation:

- The AND and OR gates behave as per standard Boolean truth tables.
- XOR produces 1 only when the two inputs differ, while XNOR produces 1 when they match.
- NAND, NOR and XNOR are correctly obtained as complements of AND, OR and XOR.

Result:

The Python program was executed successfully and all the basic and derived logic gates (AND, OR, NOT, NAND, NOR, XOR, XNOR) were correctly implemented and verified for the given inputs.

EXPERIMENT 2

Objective:

To write a Python program to control the speed of a fan according to room temperature using the concept of fuzzy logic.

Prerequisites:

- Set up Python interpreter (PyCharm).
- Basic understanding of fuzzy sets and linguistic variables.

Theory:

Fuzzy logic is a computing approach based on "degrees of truth" rather than the usual "true or false" (1 or 0) Boolean logic. In a fuzzy control system, the crisp input (room temperature) is mapped onto linguistic variables such as Low, Medium and High using membership functions, and a rule base decides the output (fan speed) accordingly.

- IF temperature is Low THEN fan speed is Low.
- IF temperature is Medium THEN fan speed is Medium.
- IF temperature is High THEN fan speed is High.

In this simplified implementation, the fuzzy regions are approximated using crisp threshold values: temperature below 18°C is treated as Low, between 18°C and 25°C as Medium, and above 25°C as High, and the corresponding fan speed is displayed.

PYTHON

```
temp = 30
print("Temp (°C):", temp)
if temp < 18:
    speed = "Low"
elif temp < 25:
    speed = "Medium"
else:
    speed = "High"
print("Fan Speed:", speed)
```

Output:

OUTPUT

```
Temp (°C): 30
Fan Speed: High
```

Observation:

- For a room temperature of 30°C (which falls in the High region), the fan speed is correctly set to High.
- The mapping correctly distinguishes between the Low, Medium and High temperature regions.

Result:

The Python program successfully simulates a fuzzy-logic-based fan speed controller that adjusts the fan speed according to the room temperature.

EXPERIMENT 3

Objective:

To write a Python program to compute the α-Cut and Strong α-Cut of any given fuzzy set.

Prerequisites:

- Set up Python interpreter (PyCharm).
- Basic knowledge of fuzzy sets and membership functions.

Theory:

A fuzzy set A defined on a universe of discourse X is characterised by a membership function μA(x) that assigns to every element x a value in the interval [0, 1].

The α-Cut of a fuzzy set A, denoted Aα, is the crisp set containing all elements of the universe whose membership grade is greater than or equal to a specified threshold α:

$$A\alpha = \{ x \in X / \mu A(x) \geq \alpha \}$$

The Strong α-Cut, denoted Aα+, contains only those elements whose membership grade is strictly greater than α:

$$A\alpha+ = \{ x \in X / \mu A(x) > \alpha \}$$

α-Cuts are useful because they convert a fuzzy set into a family of crisp (classical) sets, which simplifies analysis and decision-making in fuzzy systems.

PYTHON

```
A = {'a': 0.2, 'b': 0.5, 'c': 0.7, 'd': 1.0, 'e': 0.4}
alpha = 0.5
alpha_cut = []
strong_alpha_cut = []
for element, membership in A.items():
    if membership >= alpha:
        alpha_cut.append(element)
    if membership > alpha:
        strong_alpha_cut.append(element)
print("Fuzzy Set:", A)
print("Alpha value:", alpha)
print("Alpha-Cut:", alpha_cut)
print("Strong Alpha-Cut:", strong_alpha_cut)
```

Output:

OUTPUT

```
Fuzzy Set: {'a': 0.2, 'b': 0.5, 'c': 0.7, 'd': 1.0, 'e': 0.4}
Alpha value: 0.5
Alpha-Cut: ['b', 'c', 'd']
Strong Alpha-Cut: ['c', 'd']
```

Observation:

- For α = 0.5, elements with membership ≥ 0.5 (b, c, d) form the α-Cut.
- The Strong α-Cut excludes the element whose membership is exactly equal to α (element b), leaving only c and d.

Result:

The Python program successfully computed the α-Cut and Strong α-Cut of the given fuzzy set for a specified value of α.

EXPERIMENT 4

Objective:

To write a Python program to determine the height of a fuzzy set and classify it as Normal or Subnormal.

Prerequisites:

- Set up Python interpreter (PyCharm).
- Basic knowledge of fuzzy set theory.

Theory:

The height of a fuzzy set A, denoted h(A), is defined as the largest membership grade attained by any element in that set:

$$h(A) = max_x \, \mu A(x)$$

A fuzzy set is said to be Normal if its height is equal to 1, i.e. at least one element has full membership. If the height is less than 1, the fuzzy set is said to be Subnormal.

PYTHON

```
A = {'a': 0.2, 'b': 0.6, 'c': 0.9, 'd': 0.8}
height = max(A.values())
print("Fuzzy Set:", A)
print("Height =", height)
if height == 1:
    print("The fuzzy set is NORMAL")
else:
    print("The fuzzy set is SUBNORMAL")
```

Output:

OUTPUT

```
Fuzzy Set: {'a': 0.2, 'b': 0.6, 'c': 0.9, 'd': 0.8}
Height = 0.9
The fuzzy set is SUBNORMAL
```

Observation:

- The maximum membership value in the given fuzzy set is 0.9, which is less than 1.
- Since the height is less than 1, the fuzzy set is correctly classified as Subnormal.

Result:

The Python program successfully computed the height of the given fuzzy set and correctly classified it as Subnormal.

EXPERIMENT 5

Objective:

To write a Python program to determine the Support, Core, Boundary and Crossover points of a fuzzy set.

Prerequisites:

- Set up Python interpreter (PyCharm).
- Basic knowledge of fuzzy set theory.

Theory:

- Support: The set of all elements with non-zero membership, i.e. $\{ x \mid \mu A(x) > 0 \}$.
- Core: The set of all elements with full membership, i.e. $\{ x \mid \mu A(x) = 1 \}$.
- Boundary: The set of all elements with partial membership, i.e. $\{ x \mid 0 < \mu A(x) < 1 \}$.
- Crossover Point: Any element whose membership grade is exactly 0.5.

These regions together describe the complete shape of a fuzzy set and are frequently used while designing membership functions.

PYTHON

```
A = {'a': 0, 'b': 0.3, 'c': 0.5, 'd': 1, 'e': 0.8, 'f': 0}
support = []
core = []
boundary = []
crossover = []
for element, membership in A.items():
    if membership > 0:
        support.append(element)
    if membership == 1:
        core.append(element)
    if 0 < membership < 1:
        boundary.append(element)
    if membership == 0.5:
        crossover.append(element)
print("Fuzzy Set:", A)
print("Support:", support)
print("Core:", core)
print("Boundary:", boundary)
print("Crossover Points:", crossover)
```

Output:

OUTPUT

```
Fuzzy Set: {'a': 0, 'b': 0.3, 'c': 0.5, 'd': 1, 'e': 0.8, 'f': 0}
Support: ['b', 'c', 'd', 'e']
Core: ['d']
Boundary: ['b', 'c', 'e']
Crossover Points: ['c']
```

Observation:

- Elements a and f, having membership 0, are excluded from the Support.
- Element d, with membership 1, forms the Core of the fuzzy set.
- Elements b, c and e lie strictly between 0 and 1 and hence form the Boundary.
- Element c, with membership exactly 0.5, is identified as the Crossover Point.

Result:

The Python program successfully determined the Support, Core, Boundary and Crossover points of the given fuzzy set.

EXPERIMENT 6

Objective:

To write a Python program to convert a crisp set into a fuzzy set by assigning appropriate membership values to its elements.

Prerequisites:

- Set up Python interpreter (PyCharm).
- Basic knowledge of crisp sets and fuzzification.

Theory:

A crisp set contains elements with a well-defined boundary, where an element either fully belongs (membership = 1) or does not belong (membership = 0) to the set. Fuzzification is the process of converting such a crisp set into a fuzzy set by assigning to every element a membership grade in the interval [0, 1], representing the degree to which that element belongs to the set.

This program takes a crisp set of elements and assigns a membership value to each element, thereby constructing the corresponding fuzzy set.

PYTHON

```
crisp_set = ['a', 'b', 'c', 'd']
sample_memberships = [0.9, 0.4, 0.6, 0.1]

fuzzy_set = {}
for element, membership in zip(crisp_set, sample_memberships):
    fuzzy_set[element] = membership

print("Crisp Set:", crisp_set)
print("\nFuzzy Set:")
for element, membership in fuzzy_set.items():
    print(f"{element} : {membership}")
```

Output:

OUTPUT

```
Crisp Set: ['a', 'b', 'c', 'd']

Fuzzy Set:
a : 0.9
b : 0.4
c : 0.6
d : 0.1
```

Observation:

- Every element of the crisp set is assigned a membership degree between 0 and 1.
- Elements with a higher assigned membership value (e.g. 'a' with 0.9) can be regarded as belonging more strongly to the fuzzy set than elements with a lower value (e.g. 'd' with 0.1).

Result:

The Python program successfully converted the given crisp set into a fuzzy set by assigning membership values to its elements.

EXPERIMENT 7

Objective:

To implement Union, Intersection, Complement and Difference operations on fuzzy sets; to create a fuzzy relation using the Cartesian product of two fuzzy sets; and to perform Max-Min composition on two fuzzy relations.

Prerequisites:

- Set up Python interpreter (PyCharm).
- Knowledge of fuzzy set operations and fuzzy relations.

Theory:

For two fuzzy sets A and B defined on the same universe of discourse, the standard fuzzy set operations are defined as:

$$\mu(A \cup B)(x) = \max(\mu A(x), \mu B(x))$$

$$\mu(A \cap B)(x) = \min(\mu A(x), \mu B(x))$$

$$\mu(A^{c})(x) = 1 - \mu A(x)$$

$$\mu(A - B)(x) = \min(\mu A(x), 1 - \mu B(x))$$

A fuzzy relation R between two fuzzy sets C and D can be formed using their Cartesian product, where the membership of a pair (x, y) is given by the minimum of the two individual memberships:

$$\mu R(x, y) = \min(\mu C(x), \mu D(y))$$

Given two fuzzy relations R1 (on X × Y) and R2 (on Y × Z), the Max-Min composition R1 ∘ R2 (a relation on X × Z) is defined as:

$$\mu(R1 \circ R2)(x, z) = \max_{y} [\min(\mu R1(x,y), \mu R2(y,z))]$$

PYTHON

```
A = {'a': 0.2, 'b': 0.7, 'c': 0.9}
B = {'a': 0.6, 'b': 0.4, 'c': 0.8}

print("Fuzzy Set A:", A)
print("Fuzzy Set B:", B)

union = {}
for x in set(A) | set(B):
    union[x] = max(A.get(x, 0), B.get(x, 0))
print("\nUnion (A U B):", union)

intersection = {}
for x in set(A) | set(B):
    intersection[x] = min(A.get(x, 0), B.get(x, 0))
print("Intersection (A n B):", intersection)

complement = {}
for x in A:
    complement[x] = 1 - A[x]
print("Complement of A:", complement)

difference = {}
for x in set(A) | set(B):
    difference[x] = min(A.get(x, 0), 1 - B.get(x, 0))
print("Difference (A - B):", difference)

C = {'x': 0.5, 'y': 0.8}
D = {'p': 0.4, 'q': 0.9}
```

```
R1 = {}
for x in C:
    for y in D:
        R1[(x, y)] = min(C[x], D[y])

print("\nFuzzy Set C:", C)
print("Fuzzy Set D:", D)
print("Cartesian Product C x D (Relation R1):", R1)

R2 = {
    ('p', 'm'): 0.7,
    ('p', 'n'): 0.5,
    ('q', 'm'): 0.6,
    ('q', 'n'): 0.8
}
print("\nRelation R2:", R2)

R3 = {}
X = set(x for x, y in R1)
Y = set(y for x, y in R1)
Z = set(y for x, y in R2)

for x in X:
    for z in Z:
        values = []
        for y in Y:
            if (x, y) in R1 and (y, z) in R2:
                values.append(min(R1[(x, y)], R2[(y, z)]))
        if values:
            R3[(x, z)] = max(values)

print("Max-Min Composition R1 o R2:", R3)
```

Output:

```
OUTPUT
Fuzzy Set A: {'a': 0.2, 'b': 0.7, 'c': 0.9}
Fuzzy Set B: {'a': 0.6, 'b': 0.4, 'c': 0.8}

Union (A U B): {'c': 0.9, 'b': 0.7, 'a': 0.6}
Intersection (A n B): {'c': 0.8, 'b': 0.4, 'a': 0.2}
Complement of A: {'a': 0.8, 'b': 0.30000000000000004, 'c': 0.09999999999999998}
Difference (A - B): {'c': 0.19999999999999996, 'b': 0.6, 'a': 0.2}

Fuzzy Set C: {'x': 0.5, 'y': 0.8}
Fuzzy Set D: {'p': 0.4, 'q': 0.9}
Cartesian Product C x D (Relation R1): {('x', 'p'): 0.4, ('x', 'q'): 0.5, ('y',
'p'): 0.4, ('y', 'q'): 0.8}

Relation R2: {('p', 'm'): 0.7, ('p', 'n'): 0.5, ('q', 'm'): 0.6, ('q', 'n'):
0.8}
Max-Min Composition R1 o R2: {('y', 'm'): 0.6, ('y', 'n'): 0.8, ('x', 'm'):
0.5, ('x', 'n'): 0.5}
```

Observation:

- The Union operation retains the higher membership value for every element, while Intersection retains the lower value.
- The Complement operation correctly gives 1 minus the original membership for every element of set A.
- The Cartesian product of fuzzy sets C and D correctly forms relation R1 using the min operation.

- The Max-Min composition of R1 and R2 was successfully computed, producing a new fuzzy relation between the elements of C and Z.

Result:

The Python program successfully implemented the Union, Intersection, Complement and Difference operations on fuzzy sets, and also demonstrated the formation of a fuzzy relation using the Cartesian product along with Max-Min composition.

EXPERIMENT 8

Objective:

To create a perceptron with an appropriate number of inputs and outputs, and to train it using the fixed increment learning algorithm until no further change in weight is required.

Prerequisites:

- Set up Python interpreter (PyCharm).
- Basic knowledge of artificial neural networks and the perceptron learning rule.

Theory:

A perceptron is the simplest form of an artificial neural network, consisting of a single layer of weights connecting the inputs to an output neuron with a hard-limiting (step) activation function. It can be used to learn linearly separable Boolean functions such as AND, OR, etc.

In the fixed increment learning algorithm, the weights and bias are updated after every training sample using the rule:

$$w = w + \eta\,(t - y)\,x, \quad b = b + \eta\,(t - y)$$

where t is the target output, y is the actual (predicted) output, x is the input vector and η is the learning rate. Training continues epoch after epoch until the weights and bias no longer change, indicating that the perceptron has converged to a solution that correctly classifies every training sample.

PYTHON

```
X = [[0,0],[0,1],[1,0],[1,1]]
T = [0,0,0,1]
w=[0,0]
b=0
eta=1
epoch=0

while True:
    epoch+=1
    oldW = w.copy()
    oldB = b
    print("\nEpoch:", epoch)
    for i in range(len(X)):
        net = w[0]*X[i][0] + w[1]*X[i][1] + b
        y = 1 if net > 0 else 0
        error = T[i] - y
        w[0] = w[0] + eta*error*X[i][0]
        w[1] = w[1] + eta*error*X[i][1]
        b = b + eta*error
        print("Input:", X[i], "Target:", T[i], "Output:", y, "Weights:", w,
"Bias:", b)
    if w==oldW and b==oldB:
        break

print("\nTraining Complete")
print("Final weights:", w)
print("Final bias:", b)
print("\nTesting")
for i in range(len(X)):
    net = w[0]*X[i][0] + w[1]*X[i][1] + b
    y = 1 if net > 0 else 0
    print(X[i], "->", y)
```

Output:

OUTPUT

```
Epoch: 1
Input: [0, 0] Target: 0 Output: 0 Weights: [0, 0] Bias: 0
Input: [0, 1] Target: 0 Output: 0 Weights: [0, 0] Bias: 0
Input: [1, 0] Target: 0 Output: 0 Weights: [0, 0] Bias: 0
Input: [1, 1] Target: 1 Output: 0 Weights: [1, 1] Bias: 1

Epoch: 2
Input: [0, 0] Target: 0 Output: 1 Weights: [1, 1] Bias: 0
Input: [0, 1] Target: 0 Output: 1 Weights: [1, 0] Bias: -1
Input: [1, 0] Target: 0 Output: 0 Weights: [1, 0] Bias: -1
Input: [1, 1] Target: 1 Output: 0 Weights: [2, 1] Bias: 0

Epoch: 3
Input: [0, 0] Target: 0 Output: 0 Weights: [2, 1] Bias: 0
Input: [0, 1] Target: 0 Output: 1 Weights: [2, 0] Bias: -1
Input: [1, 0] Target: 0 Output: 1 Weights: [1, 0] Bias: -2
Input: [1, 1] Target: 1 Output: 0 Weights: [2, 1] Bias: -1

Epoch: 4
Input: [0, 0] Target: 0 Output: 0 Weights: [2, 1] Bias: -1
Input: [0, 1] Target: 0 Output: 0 Weights: [2, 1] Bias: -1
Input: [1, 0] Target: 0 Output: 1 Weights: [1, 1] Bias: -2
Input: [1, 1] Target: 1 Output: 0 Weights: [2, 2] Bias: -1

Epoch: 5
Input: [0, 0] Target: 0 Output: 0 Weights: [2, 2] Bias: -1
Input: [0, 1] Target: 0 Output: 1 Weights: [2, 1] Bias: -2
Input: [1, 0] Target: 0 Output: 0 Weights: [2, 1] Bias: -2
Input: [1, 1] Target: 1 Output: 1 Weights: [2, 1] Bias: -2

Epoch: 6
Input: [0, 0] Target: 0 Output: 0 Weights: [2, 1] Bias: -2
Input: [0, 1] Target: 0 Output: 0 Weights: [2, 1] Bias: -2
Input: [1, 0] Target: 0 Output: 0 Weights: [2, 1] Bias: -2
Input: [1, 1] Target: 1 Output: 1 Weights: [2, 1] Bias: -2

Training Complete
Final weights: [2, 1]
Final bias: -2

Testing
[0, 0] -> 0
[0, 1] -> 0
[1, 0] -> 0
[1, 1] -> 1
```

Observation:

- The weights and bias keep changing during the first few epochs as the perceptron misclassifies some patterns.
- Training converges after 6 epochs, at which point the weights and bias remain unchanged for a full pass over all input patterns.
- The final trained perceptron correctly reproduces the AND gate truth table during testing.

Result:

The perceptron was successfully trained using the fixed increment learning algorithm and, upon convergence, correctly implemented the two-input AND function.

EXPERIMENT 9

Objective:

To write a Python program to implement Linear Regression using NumPy and to visualise the fitted regression line using Matplotlib.

Prerequisites:

- Set up Python interpreter (PyCharm).
- Install NumPy and Matplotlib libraries.
- Basic knowledge of statistics and the least squares method.

Theory:

Linear regression models the relationship between an independent variable x and a dependent variable y by fitting a straight line of the form:

$$y = m\,x + c$$

where m is the slope and c is the intercept of the regression line. Using the method of least squares, the slope and intercept that minimise the sum of squared errors between the actual and predicted values can be expressed in terms of the deviations of x and y from their respective means ($\bar{x}$ and $\bar{y}$):

$$m = \Sigma(x - \bar{x})(y - \bar{y}) \,/\, \Sigma(x - \bar{x})^2 \quad , \quad c = \bar{y} - m\,\bar{x}$$

Once m and c are known, the predicted value for every x can be computed as y_pred = m·x + c, and the quality of the fit can be visualised by plotting the original data points along with the fitted regression line.

PYTHON

```
import numpy as np
import matplotlib
matplotlib.use('Agg')
import matplotlib.pyplot as plt

x = np.array([1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13])
y = np.array([2, 3, 5, 6, 10, 9, 12, 11, 15, 18, 23, 23, 28])

x_mean = np.mean(x)
y_mean = np.mean(y)

m = np.sum((x - x_mean) * (y - y_mean)) / np.sum((x - x_mean) ** 2)
c = y_mean - m * x_mean

y_pred = m * x + c

print("Slope =", m)
print("Intercept =", c)
print("Equation: y =", round(m, 2), "x +", round(c, 2))

plt.figure(figsize=(6, 4.2))
plt.scatter(x, y, color="black", s=70, label="INPUT")
plt.plot(x, y_pred, color="blue", linewidth=2, label="Regression Line")
plt.xlabel("X")
plt.ylabel("Y")
plt.title("Linear Regression with 13 Data Points")
plt.grid(True)
plt.legend()
plt.tight_layout()
plt.savefig("/home/claude/softcomputing/exp9_plot.png", dpi=150)
```

Output:

OUTPUT

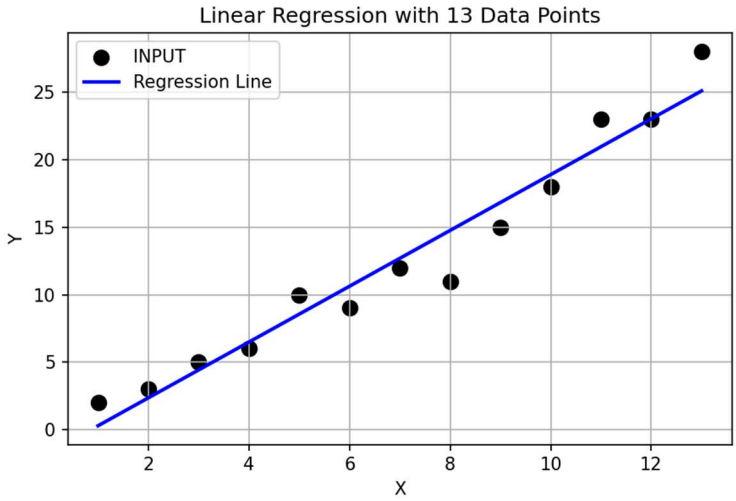
```
Slope = 2.065934065934066
Intercept = -1.7692307692307718
Equation: y = 2.07 x + -1.77
```

Observation:

- The computed slope ($m \approx 2.07$) indicates that y increases by about 2.07 units for every unit increase in x.
- The regression equation obtained is $y = 2.07x - 1.77$.
- The plotted regression line passes closely through the scattered input data points, indicating a good linear fit.

Result:

The Python program successfully implemented Linear Regression using NumPy, and the plotted graph confirmed that the fitted regression line closely follows the trend of the given data points.



EXPERIMENT 10

Objective:

To write a Python program to implement Multiple Linear Regression using NumPy, Matplotlib and scikit-learn.

Prerequisites:

- Set up Python interpreter (PyCharm).
- Install NumPy, Matplotlib and scikit-learn libraries.
- Basic knowledge of regression analysis.

Theory:

Multiple Linear Regression extends simple linear regression to model the relationship between a dependent variable y and two or more independent variables x1, x2, …, xn:

$$y = b0 + b1x1 + b2x2 + ... + bnxn$$

The coefficients b0, b1, …, bn are estimated using the least squares method so as to minimise the overall prediction error. In this program, the scikit-learn LinearRegression model is used to fit the data, and the quality of the fit is visualised by plotting actual versus predicted values.

PYTHON

```
import numpy as np
import matplotlib
matplotlib.use('Agg')
import matplotlib.pyplot as plt
from sklearn.linear_model import LinearRegression

X = np.array([
    [1,2,5],[2,5,3],[3,1,8],[4,6,4],[5,3,9],
    [6,8,2],[7,4,7],[8,7,5],[9,2,10],[10,9,3]
])
y = np.array([12,15,14,20,22,25,24,30,28,34])

model = LinearRegression().fit(X, y)
print("Coefficients:", model.coef_)
print("Intercept:", model.intercept_)
print("Prediction for [6,5,6]:", model.predict([[6,5,6]])[0])

yp = model.predict(X)
plt.figure(figsize=(5,4))
plt.scatter(y, yp, color='blue')
plt.plot([y.min(), y.max()], [y.min(), y.max()], 'r--')
plt.xlabel("Actual Y")
plt.ylabel("Predicted Y")
plt.title("Multiple Linear Regression")
plt.grid()
plt.tight_layout()
plt.savefig("/home/claude/softcomputing/exp10_plot.png", dpi=150)
```

Output:

OUTPUT

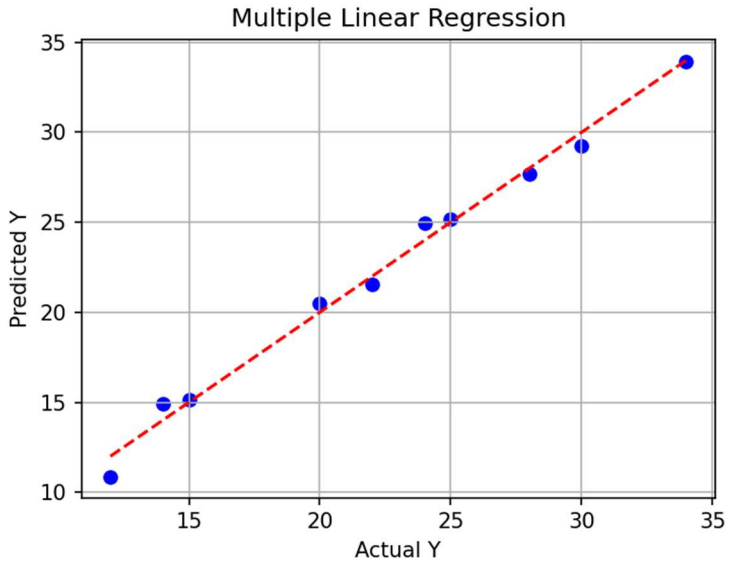
```
Coefficients: [1.69337161 1.30679983 0.65616092]
Intercept: 3.269995813647512
Prediction for [6,5,6]: 23.90119012020812
```

Observation:

- The fitted model produced coefficients of approximately [1.693, 1.307, 0.656] and an intercept of about 3.27.
- The predicted value of y for a new input [6, 5, 6] was found to be approximately 23.90.
- The scatter plot of actual versus predicted values lies close to the ideal 45° reference line, indicating a good fit.

Result:

The Python program successfully implemented Multiple Linear Regression, and the scatter plot confirmed a good agreement between the actual and predicted values.



EXPERIMENT 11

Objective:

To write a Python program to implement Hebb's Rule for the AND gate.

Prerequisites:

- Set up Python interpreter (PyCharm).
- Basic knowledge of Hebbian learning.

Theory:

Hebb's Rule is one of the earliest and simplest learning rules for artificial neural networks, based on the neuro-physiological postulate that "when one neuron repeatedly helps fire another, the connection between them is strengthened." Mathematically, the weight update is given by:

$$wi(new) = wi(old) + xi \cdot y, \quad b(new) = b(old) + y$$

where xi is the input, y is the desired (target) output, and wi and b are the weight and bias respectively. Unlike the perceptron rule, Hebbian learning does not depend on the error between target and actual output, but purely on the correlation between input and target.

PYTHON

```
inputs = [[0,0],[0,1],[1,0],[1,1]]
desired = [0,0,0,1]
w1 = 0
w2 = 0
b = 0
for i in range(len(inputs)):
    x1, x2 = inputs[i]
    y = desired[i]
    w1 = w1 + x1*y
    w2 = w2 + x2*y
    b = b + y
    print("Input:", x1, x2, "Output:", y, " w1 =", w1, " w2 =", w2, " b =", b)
print("\nFinal weights:")
print("w1 =", w1)
print("w2 =", w2)
print("Bias =", b)
```

Output:

OUTPUT

```
Input: 0 0 Output: 0  w1 = 0  w2 = 0  b = 0
Input: 0 1 Output: 0  w1 = 0  w2 = 0  b = 0
Input: 1 0 Output: 0  w1 = 0  w2 = 0  b = 0
Input: 1 1 Output: 1  w1 = 1  w2 = 1  b = 1

Final weights:
w1 = 1
w2 = 1
Bias = 1
```

Observation:

- The weights and bias remain unchanged for input patterns where the target output is 0, since Hebb's rule only updates weights when the target is non-zero.
- Only the input pattern (1, 1) with target 1 contributes to the weight update, yielding final weights w1 = 1, w2 = 1 and bias = 1.

Result:

The Python program successfully implemented Hebb's Rule and obtained the weights that represent the two-input AND function.

EXPERIMENT 12

Objective:

To write a Python program to implement the Delta Rule for the OR gate.

Prerequisites:

- Set up Python interpreter (PyCharm).
- Basic knowledge of gradient-descent-based learning rules.

Theory:

The Delta Rule (also known as the Widrow-Hoff rule or Least Mean Square rule) is a gradient-descent-based learning rule that updates weights in proportion to the error between the desired output and the actual (continuous, unthresholded) output of the neuron:

$$wi(new) = wi(old) + \eta\,(d - y)\,xi, \quad b(new) = b(old) + \eta\,(d - y)$$

where d is the desired output, y is the actual output (a linear combination of inputs and weights, without applying a threshold), and η is the learning rate. The weights are updated iteratively over several epochs so as to progressively minimise the mean squared error.

PYTHON

```
inputs = [[0,0],[0,1],[1,0],[1,1]]
desired = [0,1,1,1]
w1 = 0
w2 = 0
learning_rate = 0.1
epochs = 10
b = 0
for epoch in range(epochs):
    error = 0
    for i in range(len(inputs)):
        x1, x2 = inputs[i]
        d = desired[i]
        y = w1*x1 + w2*x2 + b
        error = d - y
        w1 = w1 + learning_rate*error*x1
        w2 = w2 + learning_rate*error*x2
        b = b + learning_rate*error
    print(f"Epoch {epoch+1}: w1={w1:.4f}, w2={w2:.4f}, b={b:.4f},
error={error:.4f}")
print("\nFinal Weights:")
print("w1 =", round(w1,4))
print("w2 =", round(w2,4))
print("b  =", round(b,4))
```

Output:

OUTPUT

```
Epoch 1: w1=0.1520, w2=0.1620, b=0.2520, error=0.6200
Epoch 2: w1=0.2305, w2=0.2456, b=0.3664, error=0.2249
Epoch 3: w1=0.2732, w2=0.2910, b=0.4149, error=0.0297
Epoch 4: w1=0.2983, w2=0.3177, b=0.4321, error=-0.0688
Epoch 5: w1=0.3146, w2=0.3350, b=0.4345, error=-0.1203
Epoch 6: w1=0.3264, w2=0.3475, b=0.4303, error=-0.1489
Epoch 7: w1=0.3358, w2=0.3574, b=0.4231, error=-0.1662
Epoch 8: w1=0.3437, w2=0.3658, b=0.4149, error=-0.1778
Epoch 9: w1=0.3507, w2=0.3732, b=0.4065, error=-0.1865
Epoch 10: w1=0.3571, w2=0.3800, b=0.3984, error=-0.1935
```

```
Final Weights:
w1 = 0.3571
w2 = 0.38
b  = 0.3984
```

Observation:

- The error progressively reduces in magnitude over the first few epochs as the weights adjust towards the target mapping.
- After 10 training epochs, the weights converge to approximately w1 = 0.357, w2 = 0.380 and bias = 0.398, providing an approximate linear fit to the AND function.

Result:

The Python program successfully implemented the Delta Rule and demonstrated the progressive reduction of error and convergence of weights over multiple training epochs.

EXPERIMENT 13

Objective:

To write a Python program to solve the Travelling Salesman Problem (TSP) using a Genetic Algorithm.

Prerequisites:

- Set up Python interpreter (PyCharm).
- Basic knowledge of genetic algorithms and the Travelling Salesman Problem.

Theory:

The Travelling Salesman Problem asks for the shortest possible route that visits every city exactly once and returns to the starting city. It is an NP-hard combinatorial optimisation problem, so for a large number of cities exhaustive search becomes impractical. A Genetic Algorithm (GA) is a population-based search technique inspired by natural selection, and is used to find good (near-optimal) tours efficiently. The main components of the GA used in this program are:

- Chromosome: a route, represented as a permutation of the city indices.
- Fitness: the reciprocal of the total tour length, so that shorter routes have higher fitness.
- Selection: the population is sorted by fitness and parents are chosen from the fitter half; the two best routes are carried over unchanged (elitism).
- Crossover: a segment of one parent is copied into the child and the remaining cities are filled in the order in which they appear in the second parent, so that no city is repeated.
- Mutation: with a small probability, two cities of a route are swapped, which maintains diversity in the population.

PYTHON

```
import random

# Distance matrix between 5 cities
distance = [
    [0, 10, 15, 20, 25],
    [10, 0, 35, 25, 17],
    [15, 35, 0, 30, 20],
    [20, 25, 30, 0, 18],
    [25, 17, 20, 18, 0]
]

cities = list(range(len(distance)))
population_size = 50
generations = 200
mutation_rate = 0.1

def fitness(route):
    total = 0
    for i in range(len(route)):
        total += distance[route[i]][route[(i + 1) % len(route)]]
    return 1 / total

def create_route():
    return random.sample(cities, len(cities))

def crossover(parent1, parent2):
    start, end = sorted(random.sample(range(len(cities)), 2))
    child = [None] * len(cities)
    child[start:end] = parent1[start:end]
    remaining = [city for city in parent2 if city not in child]
    j = 0
    for i in range(len(child)):
        if child[i] is None:
            child[i] = remaining[j]
            j += 1
    return child

def mutate(route):
    if random.random() < mutation_rate:
        i, j = random.sample(range(len(route)), 2)
        route[i], route[j] = route[j], route[i]
    return route
```

```
population = [create_route() for _ in range(population_size)]

for generation in range(generations):
    population.sort(key=fitness, reverse=True)
    new_population = population[:2]  # Keep the best routes
    while len(new_population) < population_size:
        parent1, parent2 = random.choices(
            population[:population_size // 2], k=2)
        child = crossover(parent1, parent2)
        child = mutate(child)
        new_population.append(child)
    population = new_population

best_route = max(population, key=fitness)

total_distance = sum(
    distance[best_route[i]][best_route[(i + 1) % len(best_route)]]
    for i in range(len(best_route))
)

print("\nBest Route:")
print(" -> ".join(str(city + 1) for city in best_route)
      + " -> " + str(best_route[0] + 1))
print("Minimum Distance Found:", total_distance)
```

Output:

```
OUTPUT
Best Route:
5 -> 3 -> 1 -> 2 -> 4 -> 5
Minimum Distance Found: 88
```

Observation:

- The program uses a population of 50 routes, evolved for 200 generations with a mutation rate of 0.1, on a 5-city distance matrix.
- The best route found is $5 \rightarrow 3 \rightarrow 1 \rightarrow 2 \rightarrow 4 \rightarrow 5$ (returning to city 5) with a total distance of 88.
- The result was cross-checked by exhaustive search of all possible tours, which also gives a minimum distance of 88, so the GA has found the optimal tour.
- Since the algorithm uses random initialisation, crossover and mutation, the route may appear in a different (equivalent) order or direction on different runs, but its length remains the same.

Result:

The Python program successfully solved the Travelling Salesman Problem for the given set of cities using a Genetic Algorithm and obtained the minimum tour distance of 88.

EXPERIMENT 14

Objective:

To implement an Artificial Neural Network (ANN) using a single-layer perceptron without backpropagation and to verify its operation using the AND logic gate.

Prerequisites:

- Set up Python interpreter (PyCharm).
- Basic knowledge of artificial neural networks, activation functions and the perceptron learning rule.

Theory:

A single-layer perceptron consists of an input layer connected directly to one output neuron through adjustable weights. The neuron computes the net input as the weighted sum of its inputs plus a bias and passes it through a step (hard-limiting) activation function:

$$net = w1 \cdot x1 + w2 \cdot x2 + b \text{ , } y = 1 \text{ if } net \geq 0, \text{ otherwise } y = 0$$

Since there is no hidden layer, no backpropagation is required. The weights and bias are corrected directly after every training sample using the perceptron learning rule:

$$w = w + \eta\,(t - y)\,x \text{ , } b = b + \eta\,(t - y)$$

where t is the target output, y is the actual output and η is the learning rate. Training continues for a fixed number of epochs or until an entire epoch produces no errors. As the AND function is linearly separable, the perceptron is guaranteed to converge.

PYTHON

```
inputs = [[0,0],[0,1],[1,0],[1,1]]
targets = [0,0,0,1]

w1 = 0.0
w2 = 0.0
bias = 0.0
learning_rate = 0.1
epochs = 100

def activation(net):
    return 1 if net >= 0 else 0

for epoch in range(epochs):
    errors = 0
    for x, target in zip(inputs, targets):
        net = w1*x[0] + w2*x[1] + bias
        output = activation(net)
        error = target - output
        w1 += learning_rate*error*x[0]
        w2 += learning_rate*error*x[1]
        bias += learning_rate*error
        if error != 0:
            errors += 1
    if errors == 0:
        print("Converged at epoch", epoch + 1)
        break

print("\nAND Gate Results")
print("X1 X2 Target Output")
for x, target in zip(inputs, targets):
    net = w1*x[0] + w2*x[1] + bias
    output = activation(net)
    print(x[0], x[1], " ", target, "    ", output)

print("\nFinal weights:", w1, w2)
print("Final bias:", bias)
```

Output:

```
OUTPUT
Converged at epoch 4

AND Gate Results
X1 X2 Target Output
0 0   0      0
0 1   0      0
1 0   0      0
1 1   1      1

Final weights: 0.2 0.1
Final bias: -0.20000000000000004
```

Observation:

- The weights and bias are initialised to zero, with a learning rate of 0.1 and a maximum of 100 epochs.
- The perceptron made no errors in the fourth epoch, so training stopped (converged) at epoch 4, well before the epoch limit.
- The final weights w1 = 0.2, w2 = 0.1 and bias ≈ −0.2 correctly reproduce the AND truth table: only the input (1, 1) gives output 1 (the bias value −0.20000000000000004 is a floating-point representation of −0.2).

Result:

The single-layer perceptron was successfully trained without backpropagation and correctly implemented the AND gate, as verified by the truth table obtained.